1.1 什么是OpenClaw：个人AI助手的全新范式

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~140k tokens，输出 ~3.2k tokens（本节）

很多人第一次听到 OpenClaw，脑子里会先冒出一个问题：它和 ChatGPT、Claude 这种聊天工具，到底差在哪？如果只看表面，它们都能聊天、能写东西、能回答问题，似乎差不多。但 OpenClaw 想做的，不只是“一个聊天窗口里的 AI”。它更像是一个长期住在你自己设备上的个人 AI 助手：你可以通过不同的消息入口找到它，它能记住你平时的习惯，能调动工具替你做事，而且核心控制权在你手里，不是在某家平台的后台里。

这一章是全书的第一章，所以我们先不钻源码，不急着谈复杂实现。先把最关键的感觉建立起来：OpenClaw 是什么，它为什么会出现，它适合什么人，以及它背后的两条设计主线到底是什么意思。

1.1.1 OpenClaw 不是另一个聊天网页

如果你平时主要接触的是 ChatGPT、Claude、Gemini 这一类产品，你已经很熟悉一种典型使用方式：打开网页或 App，输入一句话，等模型回你。这种方式当然很方便，但它有一个明显前提：你是在“使用一个别人搭好的 AI 服务”。

OpenClaw 则更像是在搭建“你自己的 AI 助手系统”。这里有两个关键词。

第一个关键词是“个人”。OpenClaw 默认就是为一个具体的人服务的，不是面向一个公司的全部员工，也不是一个公开的机器人平台。它更像你的私人助理，而不是一个大型 SaaS 产品里的公共功能。

第二个关键词是“助手”。聊天只是入口，重点不是把问题答出来，而是接住事情、继续做事。比如：

- 你在 Telegram 上丢一句“明早 8 点提醒我交作业”，它不只是回复“好的”，还可以真的安排提醒。
- 你在 WhatsApp 上发一张截图，让它帮你整理成待办，它可以继续调用工具处理内容。
- 你在电脑上写代码写到一半，直接让它读项目、跑命令、改文件，它不需要你把上下文重新复制一遍。

也就是说，OpenClaw 不把自己定位成“一个对话网站”，而是“一个可长期运行、可持续协作的个人 AI 系统”。这个差别，决定了它后面很多设计都和常见聊天产品不一样。

1.1.2 它为什么会出现

OpenClaw 出现，不是因为世界上缺少一个聊天机器人，而是因为很多重度用户很快会碰到现有产品的边界。

先看几个常见痛点。

第一，你的入口太分散。你可能在 Slack 里聊工作，在 Telegram 里记灵感，在手机上看通知，在电脑终端里写代码。传统 AI 产品往往要求你主动“打开它”，而不是让它自然地出现在你已经在用的环境里。

第二，你的上下文不连贯。很多时候，你在一个地方说过的话，到另一个地方就断了。今天在网页里讲过的需求，明天到手机上又得重说一遍。对轻度使用者来说，这只是麻烦；对长期协作来说，这会直接拉低效率。

第三，控制权不在你手里。你的配置、记录、行为方式、工具权限，很多都取决于产品方怎么设计。你能不能定制，能定制到什么程度，经常不是技术问题，而是产品边界问题。

第四，AI 很会“说”，但不一定很会“干”。如果一个系统只能在聊天框里输出文字，那它更像咨询顾问；如果它能连上你的通道、工具、设备和日常工作流，它才更像助理。

OpenClaw 就是在这个背景下被设计出来的。它试图回答一个很直接的问题：能不能把 AI 从“一个云端网页”变成“一个在你自己环境里长期在线、随叫随到、能动手做事的个人助手”？

这也是为什么它会强调消息通道、工具系统、记忆、长期运行这些能力。因为这些东西拼起来，才更接近“助手”，而不只是“问答器”。

1.1.3 它适合谁，不适合谁

OpenClaw 很吸引人，但它不是给所有人准备的。

它最适合下面几类人：

- 愿意折腾工具链的开发者和技术爱好者。
- 希望把 AI 接进自己日常工作流的人，比如用消息软件、终端、脚本、自动化工具串起来做事。
- 对隐私、控制权、可定制性比较在意的人。
- 想研究“一个个人 AI 助手系统是怎么搭起来的”的学生和工程师。

如果你只是偶尔问几个问题、写几段文案、查一些资料，其实直接用现成的聊天产品就足够了。OpenClaw 的门槛比“打开网页就能用”高不少，它需要你理解配置、运行环境、模型接入，后面还会接触网关、通道、技能这些概念。

换句话说，它更像一套可编排的基础设施，而不是一个开箱即用、什么都替你包好的消费级 App。

但也正因为这样，它很适合拿来学习。你不只是“用了一个 AI 产品”，而是在逐步看懂：一个真正能工作的 AI 助手，背后需要哪些模块，为什么要这么设计，系统边界又在哪里。

1.1.4 和 ChatGPT 这类产品比，差别到底在哪

这里最好别把 OpenClaw 和 ChatGPT 简单理解成“谁更强”。它们解决的问题并不完全一样。

ChatGPT、Claude 这一类产品的长处，是把模型能力包装得非常顺手。注册、登录、打开、提问，几乎没有学习成本。它们更像成熟的 AI 服务入口。

OpenClaw 的长处，则在于“你可以把 AI 放进自己的地盘里”。具体看，有几个明显区别。

第一，入口不同。传统产品一般以官方网页和 App 为中心；OpenClaw 可以把 Telegram、WhatsApp、Slack、网页聊天、终端等入口汇到一起。你不是必须进入某个专用界面，很多时候你就在原来的沟通环境里直接找它。

第二，角色不同。传统产品默认是“一个服务”；OpenClaw 默认是“你的助手实例”。这个实例跟着你走，配置、记忆、工作区、习惯都围绕你展开。

第三，可定制程度不同。传统产品当然也有插件、GPTs、Projects 之类机制，但边界依旧由平台给定。OpenClaw 则更像是开放底座，你能接通道、配模型、装技能、改行为，还能延伸到自己的设备和脚本环境。

第四，持续在线的感觉不同。你可以把 OpenClaw 理解成一个长期开着的服务。有人给你发消息，它能接住；定时任务到了，它能执行；你从另一个入口切过来，它还能延续上下文。这种“常驻”的感觉，和“临时打开一个聊天页面”很不一样。

当然，OpenClaw 并不意味着完全脱离云端模型。它依然经常会调用外部大模型服务来做推理。区别不在于“完全本地跑大模型”，而在于“整个助理系统的控制层和使用方式由你掌握”。这个点很重要。

1.1.5 什么叫 local-first

OpenClaw 很强调一个词：local-first，也就是“本地优先”。这个词听起来有点技术味，但意思其实不绕。

所谓本地优先，不是说所有东西都必须离线完成，更不是说绝不访问网络。它真正强调的是：你的核心数据、配置和控制权，优先掌握在你自己手里。

放到 OpenClaw 里，可以把它理解成三层意思。

第一层，你的助手运行在你的设备或你自己控制的机器上。它不是某个平台替你托管的一段黑箱逻辑。你知道它在哪里运行，也知道它依赖什么。

第二层，和你有关的很多关键内容放在本地。比如配置、工作区、长期记忆、会话记录、技能说明等，原则上都更接近你的文件系统，而不是只存在于某个你看不见的后台面板里。

第三层，哪怕模型推理本身常常还要调用外部服务，整个“调度和组织过程”依旧是你自己的。该用哪个模型、谁能找这个助手、它能动哪些工具、保留哪些记忆，这些不是平台预设死的，而是你能决定的。

如果你以前没接触过 local-first，可以把它想成这样：不是你把自己交给某个 AI 平台，而是你把模型能力接进自己的系统里。前者像“租用一个现成办公室”，后者像“在自己房子里搭工作间”。两者都能办公，但掌控感完全不同。

1.1.6 什么叫 single-user

OpenClaw 还有一条很鲜明的设计路线：single-user，也就是“单用户优先”。

这里的“单用户”不是说系统只能认一个聊天对象，也不是说别人永远不能和你的助手互动。它更准确的意思是：这套系统默认是围绕一个主人来设计的，很多简化和优化都建立在这个前提上。

为什么这样做？因为一旦你要做成一个多租户系统，也就是同时服务很多相互隔离的用户，事情马上就复杂起来。你要考虑账号体系、权限边界、配额分配、数据隔离、计费、安全审批、团队协作规则……这些当然也重要，但它们解决的是“平台产品”问题，不是“个人助理”问题。

OpenClaw 选择先把“一个人的助手”做到位。这样带来的好处很实际：

- 配置简单，不需要先搭一整套组织管理系统。
- 决策直接，很多默认行为都能按“这是你自己的机器和助手”来设计。
- 体验更连续，助手不需要在不同用户之间频繁切换人格、权限和上下文。
- 工程上更容易把精力放在通道、工具、记忆、自动化这些真正影响助理体验的地方。

你可以把它理解成：OpenClaw 不是先造一座商场，再给你分一个柜台；它更像先给你造一个私人工作室。工作室没那么“企业级”，但非常贴身，也更灵活。

1.1.7 一个更像“人”的使用场景

为了把前面的概念连起来，我们看一个真实感更强的使用场景。

假设你是个计算机专业学生，平时既写代码，也做课程项目，还常常在不同设备之间切换。你可以这样用 OpenClaw：

- 在手机上的 Telegram 里，随手把老师布置的任务发给它，让它整理成待办。
- 回到电脑后，在终端里继续让它帮你梳理项目计划，甚至直接辅助你改代码。
- 晚上想到一个点子，又在另一个消息入口里补充一句，它还能接着前面的上下文往下走。
- 到了第二天早上，定时任务提醒你今天有答辩，顺手再把准备材料整理一遍。

这时候你会发现，OpenClaw 的价值不在“它回答得多华丽”，而在“它像一个持续在线的工作搭子”。入口可以变，设备可以变，任务形态也可以变，但背后还是同一个助手。

这也是为什么本书后面会花很多篇幅讲消息通道、网关、会话、工具、技能、记忆。因为这些东西不是零散功能，它们共同服务的目标，就是把 AI 从一次性问答，变成长期协作。

1.1.8 学这一章时，你应该抓住什么

如果你现在还是第一次接触 OpenClaw，其实不用急着记所有术语。先抓住三件事就够了。

第一，OpenClaw 的核心目标不是做一个更花哨的聊天界面，而是做一个属于你自己的个人 AI 助手系统。

第二，它最重要的设计味道是“本地优先”和“单用户优先”。前者强调控制权，后者强调场景聚焦。你后面看到的很多实现选择，基本都能从这两条线索里找到原因。

第三，OpenClaw 的强项在于把多个入口、工具能力和长期上下文接起来。也正因为如此，它天然比普通聊天产品更像一套工程系统。

理解了这三点，后面你再看架构，就不会觉得那些模块只是“技术名词堆在一起”。你会知道，它们其实都在服务同一件事：让 AI 真正变成一个能长期陪你工作的助手。

本节小结

1. OpenClaw 不是单纯的聊天工具，更像运行在你自己环境里的个人 AI 助手系统。
2. 它出现的原因，是很多重度用户不满足于“只会聊天的 AI”，而是希望 AI 真能接入日常工作流去做事。
3. 和 ChatGPT 这类产品比，OpenClaw 更强调入口整合、长期在线、可定制和个人控制权。
4. local-first 可以理解为“核心数据和控制层优先放在你手里”，不是简单等于“完全离线”。
5. single-user 指的是系统默认围绕一个主人设计，先把个人助手场景做到顺手，而不是先做成多租户平台。